Architectures

- Monolith
- Microservices
- Architecture hexagonale
- Clean Architecture (propre + structuré)
- Domain-Driven Design (DDD) (modéliser avec le métier)
- Architecture en couches classique (layered)

Monolith

> **C'est une seule grosse application**.
> Tout est dans un seul projet : les pages web, le code métier, la base de données.

- Facile à démarrer.
- Difficile à maintenir si ça devient trop gros

Exemple:

- Projet Spring Boot avec `controller`, `service`, `repository` dans le même code
- Un seul `.jar`

Exemple de monolithe hexagonal

```
src/
├── domain/                <-- logique métier pure
│   ├── model/
│   ├── service/
│   └── repository/        <-- interfaces
├── infrastructure/        <-- JPA, Email, API externes
│   ├── persistence/
│   └── api/
├── application/           <-- use cases, orchestration
├── web/                   <-- REST Controllers
└── MainApplication.java
```

Microservices

> Tu divises ton application en **petits services indépendants**.
>
> Chaque service a **son code, sa base, son rôle**.

- Chacun peut évoluer, être déployé seul.
- Plus complexe à gérer (réseaux, sécurité, coordination)

Architecture hexagonale

L'architecture hexagonale, aussi appelée **Ports and Adapters**, vise à **isoler la logique métier** du reste de l'application, comme les bases de données, les APIs REST, les messages Kafka, etc.

Au **centre**, on trouve le **domaine métier** (entités, règles métier, cas d'usage), qui est **pur**, sans dépendance à un framework ou outil technique (comme Spring ou JPA).

Autour de ce noyau, il y a des **ports**, c'est-à-dire des interfaces d'entrée et de sortie :

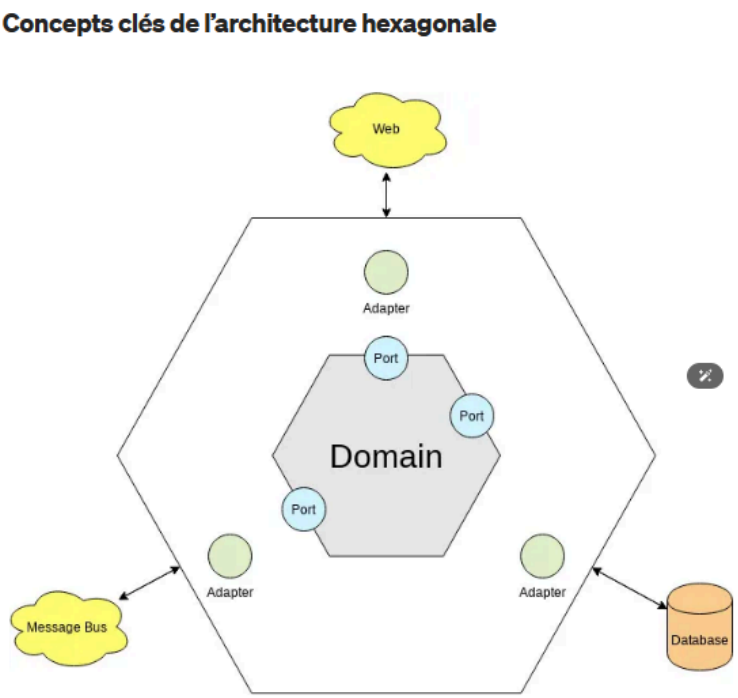
- **Ports d'entrée** : pour les actions qu'on peut exécuter sur le système (ex : use cases).
- **Ports de sortie** : pour les dépendances externes (base de données, email, etc).

Puis, les **adapters** viennent **implémenter les ports**, et connectent le domaine à l'extérieur (ex : un contrôleur REST, ou un repository JPA).

Avantage : la **logique métier reste testable, stable et indépendante** des technos ou du type d'I/O.

> Tu **protèges ton code métier** en le mettant au **centre** du projet.
> Le reste (web, DB, mails, API) est branché autour comme des **adaptateurs**.

Objectif : ne jamais mettre de `@Entity`, `@RestController` ou `@Autowired` dans le code métier pur.



Clean Architecture (propre + structuré)

> C'est une **architecture en cercles** où :

- Le **métier est au centre**
- Les **use cases** autour
- Puis les **adapters techniques** à l'extérieur

C'est un **modèle d'organisation du code** proposé par **Robert C. Martin (Uncle Bob)**.
L'objectif est de **séparer clairement les responsabilités** dans l'application, **mettre le métier au centre**, et **rendre les dépendances unidirectionnelles** (de l'extérieur vers le cœur).

Le code **du centre ne dépend jamais** des couches externes.

```

╔═══════════════╗
║  Controller   ╒ ← dépend du use case
╚═══════════════╝
╔═══════════════╗
║ Use Case      ╒ ← dépend du domaine
╚═══════════════╝
╔═══════════════╗
║ Domaine pur   ╒ ← dépend de rien
╚═══════════════╝

```

Plus tu vas vers le centre, plus le code est **pur** (aucune dépendance à des libs/frameworks).

Que veut dire "code pur" ?

Du **code métier** qui :

- n'a **aucune dépendance** à Spring, Hibernate, JPA, etc.
- ne fait **pas de** `@Autowired`, `@Entity`, `@RestController`, `@Service`
- ne lance **aucune requête HTTP ou SQL**
- est 100% **testable sans mock technique**

Microservice non-hexagonal (mauvais style) :

```java
@RestController
public class ClientController {
    @Autowired
    private ClientRepository repo;

    @GetMapping("/clients")
    public List<Client> getClients() {
        return repo.findAll(); // appel direct JPA ici
    }
}
```

- Le contrôleur connaît directement la base. Pas de séparation. C'est **couplé**.

Microservice hexagonal (bon style) :

```java
// Domaine pur
@Entity
@Data
public class Client {
    private String nom;
    private Adresse adresse;
}

// Domaine pur
@Data
public class Adresse {
    private final String valeur;
}

// Port
public interface ClientRepository {
    List<Client> findAllClients();
}

// Use Case
public class ListClientsUseCase {
    private final ClientRepository repository;

    public ListClientsUseCase(ClientRepository repository) {
        this.repository = repository;
    }

    public List<Client> handle() {
        return repository.findAllClients();
    }
}

// Adapter REST
@RestController
public class ClientController {
    private final ListClientsUseCase useCase;

    public ClientController(ListClientsUseCase useCase) {
        this.useCase = useCase;
```

```
    }

    @GetMapping("/clients")
    public List<Client> getClients() {
        return useCase.handle();
    }
}
```

Dans l'architecture hexagonale :

- Un **port** est une **interface** définie par le **domaine** ou les **use cases**. Il définit **ce que le système veut faire**, **sans dire comment**.
  - Exemple : `ClientRepository` ou `EnvoyerNotificationPort`
- Un **port d'entrée** est une **interface exposée par le domaine**, qui représente une **action métier** que l'extérieur peut appeler.
  - Exemple : `CréerClientUseCase`, `EnvoyerNotificationUseCase`
  - Il est appelé par des **adapters entrants** (comme un `@RestController`)
- Un **port de sortie** est une **interface définie par le domaine**, pour exprimer un **besoin d'accès à une ressource externe**.
  - Exemple : `ClientRepository`, `NotificationSenderPort`
  - Il est implémenté par des **adapters sortants** (comme un `JpaRepository`, un `EmailSender`)
- Un **adapter** est une **implémentation concrète d'un port**, souvent dépendante de la technologie.
  - Exemple : un `JpaRepository` qui implémente `ClientRepository`, ou un `@RestController` qui appelle un use case.

Comment tester un Use Case dans une architecture hexagonale sans dépendre de la base de données ni de Spring ?

Dans une architecture hexagonale, les **use cases** (cas d'usage) sont **découplés de la base de données** grâce aux **ports** (interfaces).

Pour les tester :

- On crée une **fausse implémentation du port de sortie** (ex : un `FakeClientRepository` qui utilise une liste en mémoire).
- Ensuite, on instancie directement le use case avec cette fausse implémentation.
- Enfin, on appelle la méthode métier (`handle()`, `execute()`...) et on vérifie le résultat avec des assertions.

Domain-Driven Design (DDD) (modéliser avec le métier)

> DDD = réfléchir en métier, coder en langage métier, et organiser le projet comme parle le client.

DDD veut dire : tu construis ton code autour du métier réel.

Autrement dit :

- Tu parles avec les gens du métier (les vrais utilisateurs)
- Tu **utilises leurs mots** dans ton code (ex : Client, Facture, Commande)
- Tu **organises ton projet** autour de la **logique métier**, pas autour de la base de données ou des écrans

Concept	Idée principale
Monolithe	Tout est dans 1 seul projet
Microservices	Tu découpes en petits services
Hexagonale	Le métier est isolé des technos
Clean Architecture	Le métier est au centre, structuré en cercles
DDD	Tu modélises ton code comme parle le client métier

Il existe des **microservices hexagonaux et** des **microservices non-hexagonaux**.
Le **mot "microservice"** parle de **la taille et l'autonomie** du service.
Le **mot "hexagonal"** parle de **la manière dont tu structures ton code**.

Architecture en couches classique (layered)

L'architecture en couches classique (layered) est organisée en niveaux hiérarchiques :

- Controller (Web) en haut,
- Service (logique métier) au milieu,
- Repository (accès base de données) en bas.

Chaque couche dépend de la couche inférieure. Le problème est que le métier dépend souvent directement de la base de données ou des technos (Spring, JPA).

En revanche, dans l'architecture hexagonale, on inverse les dépendances :

Le métier est au centre, et ne dépend de rien.

On définit des interfaces (ports) dans le domaine.

Les technologies (REST, DB, Kafka…) sont branchées via des adapters, en périphérie.

Ainsi, c'est le monde extérieur qui dépend du métier, et pas l'inverse. Cela rend l'application plus modulaire, testable et durable.

Structure de dossiers recommandée (Clean + Hexagonal)

```
src
└── main
    └── java
        └── ma.pca.notification
            ├── domain
            │   ├── model
            │   │   └── Notification.java          ← 100% POJO, pas de @Entity
            │   ├── port
            │   │   ├── in                         ← interfaces des Use Cases
            │   │   └── out                        ← interfaces des accès DB
            │   └── usecase
            │       └── SendNotificationUseCase.java
            │
            ├── application
            │   └── service
            │       └── NotificationService.java   ← impl de use case, utilise port.out
            │
            ├── adapter
            │   ├── in
            │   │   └── rest
            │   │       └── NotificationController.java
            │   └── out
            │       └── jpa
            │           ├── NotificationEntity.java  ← ici @Entity
            │           └── NotificationJpaRepository.java
            │
            └── config
                └── BeanConfig.java
```